

Project Report on

PayShield: A Smart Contract–Driven Payment Security Platform

Submitted in partial fulfillment of the
degree of Bachelor of Technology in
Computer Science and Technology

Submitted by :

Tanisha Bhomkar (07)

Mansi Satpute (66)

Akshata Ubhale (74)

Under the guidance of

Miss Shehnaz Siddique



Usha Mittal Institute of Technology

Juhu Tara Road, Santacruz (W),

Mumbai, Maharashtra

2025–2026

Declaration

I, **Tanisha Bhomkar, Mansi Satpute and Akshata Ubhale**, hereby declare that the work presented in this project entitled PayShield: A Smart Contract–Driven Payment Security Platform is entirely my own. The content of this project has been generated through my independent efforts, research, and scholarly contributions. I further declare that:

1. **Originality:** The ideas, concepts, and contributions presented in this work are solely the result of my own intellectual endeavours.
2. **Authenticity:** All data, figures, tables, and findings presented in this project are genuine and have not been fabricated or manipulated.
3. **No Use of AI Tools:** I have not used any AI-based tools to generate significant portions of this project including content, research objectives, hypotheses, and analysis.
4. **No Plagiarism:** I have properly cited and referenced all external sources and works consulted during the preparation of this project. There is no instance of plagiarism or unauthorized use of others' intellectual property.
5. **Independent Work:** This work has been conducted independently, without collaboration or assistance compromising the originality of the content.
6. **Academic Integrity:** I have adhered to the principles of academic integrity and ethical research throughout the production of this project.

I understand the consequences of academic dishonesty and affirm that this declaration accurately reflects the nature and authenticity of my work.

Date: _____

Signature: _____

[Tanisha Bhomkar]

Signature: _____

[Mansi Satpute]

Signature: _____

[Akshata Ubhale]

CERTIFICATE

This is to certify that **Tanisha Bhomkar(07)** , **Mansi Satpute(66)** , **Akshata Ubhale(74)** has completed the Project Phase 2 report on the topic ” **PayShield: A Smart Contract–Driven Payment Security Platform**” satisfactorily in partial fulfillment for the Bachelor’s Degree in Computer Science and Technology under the guidance of **Miss.Shehnaz Siddique** during the year 2025-26 as prescribed by **Usha Mittal Institute of Technology, SNDT Women’s University**.

Guide

Head Of Department

Miss Shehnaz Siddique

Prof.Kumud Wasnik

Principal

Name of Principal

Examiner 1

Examiner 2

Acknowledgment

We would like to express our sincere appreciation to everyone who provided invaluable assistance and support throughout the course of this project. We are deeply grateful to our guide, **Miss. Shehnaz Siddique**, for her continuous guidance, encouragement, and insightful feedback, which have been instrumental in the successful completion of this work.

We also extend our gratitude to the **Usha Mittal Institute of Technology, SNDT Women's University**, for providing us with the facilities and environment necessary to carry out this project.

Our heartfelt thanks go to our classmates and friends for their valuable feedback and cooperation during the development process. Finally, we would like to thank our families for their constant encouragement and support.

Tanisha Bhomkar (07)

Mansi Satpute (66)

Akshata Ubhale (74)

Abstract

Digital payment systems have become an essential part of modern financial transactions, but they are increasingly vulnerable to fraud, unauthorized access, and cyber threats. Traditional security mechanisms often fail to provide real-time protection and transparency for users. To address these challenges, this project proposes PayShield, a secure digital wallet transaction system that leverages advanced technologies to enhance transaction safety and user trust. The system integrates machine learning to detect suspicious transaction patterns and potential fraud in real time, ensuring proactive risk mitigation. Additionally, blockchain technology is used to securely store transaction records in a decentralized and tamper-proof manner, providing transparency and data integrity. Users can perform transactions through a web-based interface, where each transaction is analyzed and verified before completion. PayShield aims to provide a secure, efficient, and user-friendly platform for digital payments without requiring complex technical knowledge. By combining artificial intelligence with blockchain, the system enhances the accuracy of fraud detection, ensures secure record-keeping, and builds a reliable financial ecosystem for users. This approach significantly improves transaction security while maintaining speed and convenience in digital payments.

Contents

List of Tables	1
List of Figures	2
Nomenclature	1
1 Introduction	2
1.1 Introduction	2
1.2 Problem Statement	2
1.3 Existing System and Proposed System	3
1.3.1 System Architecture	4
1.3.2 System Architecture Overview	5
2 Literature Survey and Implementation	8
2.1 Literature Survey Table	8
2.2 Implementation	10
2.2.1 Frontend Application & Web3 Client Portal	10
2.2.2 Off-Chain Application Server Layer	11
2.2.3 Database Architecture & Off-Chain Indexing Schema	11
2.2.4 Smart Contract Layer (Anchor/Rust Program)	12
2.2.5 Escrow Payout, Payment Security, & Storage Integration	12
2.2.6 Performance Optimization, Caching, & Administration	13
3 Software Requirements and Frontend Development	15
3.1 Tech Stack Used	15
3.1.1 Software Requirements	15
3.1.2 Hardware Requirements	16

3.2	Operating System Environment	16
3.2.1	Hardware Requirements	16
3.2.2	Software Requirements	16
4	Conclusion and Future Scope	17
4.1	Conclusion	17
4.2	Future Scope	17
4.3	References	17

List of Tables

2.1 Summary of Reviewed Literature	9
--	---

List of Figures

1.1	System Block Diagram for Existing System	3
1.2	System Block Diagram for PayShield: Integrating Web3 Escrow with AI Dispute Resolution	4
2.1	Landing page of PayShield	13
2.2	Job Prediction page of PayShield	14
2.3	Client Dashboard page of PayShield	14

Nomenclature

Term	Description
AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheets
DApp	Decentralized Application
EVM	Ethereum Virtual Machine
HTML	HyperText Markup Language
HTTPS	HyperText Transfer Protocol Secure
IPFS	InterPlanetary File System
JWT	JSON Web Token
RPC	Remote Procedure Call
Solana	High-performance blockchain platform
TLS	Transport Layer Security
UI	User Interface
USDC	USD Coin Stable Cryptocurrency
Web3	Decentralized Blockchain-based Web

1. Introduction

1.1 Introduction

The global gig economy has experienced exponential growth, driven by a shift toward flexible work arrangements and cross-border collaboration. Traditional marketplaces like Upwork, Fiverr, and Freelancer have dominated this space by connecting talent with opportunity. However, these platforms operate as centralized intermediaries, exerting total control over payment governance, dispute resolution, and fund custody.

PayShield proposes a novel financial infrastructure designed to bridge the gap between traditional freelance workflows and decentralized security. By leveraging EVM-compatible smart contracts for automated escrow and fiat-pegged stablecoins (USDC) for volatility-free settlement, PayShield eliminates the need for fallible intermediaries. Unlike standard decentralized applications that often alienate non-technical users, PayShield integrates **Account Abstraction** and **AI-driven dispute resolution**, offering the robust security of Web3 with the seamless usability of Web2. This project presents a scalable standard for secure, transparent, and instant cross-border freelance settlements.

1.2 Problem Statement

Despite the digital nature of modern work, the underlying financial systems remain fragmented and outdated. The primary issues addressed by PayShield include:

- **Asymmetric Risk for Freelancers:** Freelancers, particularly in developing economies, face exclusionary banking practices where accounts are frozen or payments delayed for weeks. They often perform labor upfront only to face chargeback fraud where clients reverse payments after work is delivered.
- **Quality and Capital Risk for Clients:** Clients hiring internationally often face the upfront payment trap. Without a transparent, neutral escrow process, they have little recourse if deliverables are poor or incomplete.
- **Centralized Inefficiency:** Existing platforms act as single points of failure. High commission fees (often 10–20%), slow settlement periods, and opaque dispute resolution processes erode trust and reduce the actual earnings of the global workforce.

1.3 Existing System and Proposed System

The existing system relies on **Centralized Finance (CeFi)** and traditional banking rails (SWIFT), which are plagued by high fees and slow processing times. Disputes are handled by platform-specific administrators whose decisions can be arbitrary and lack cryptographic proof.

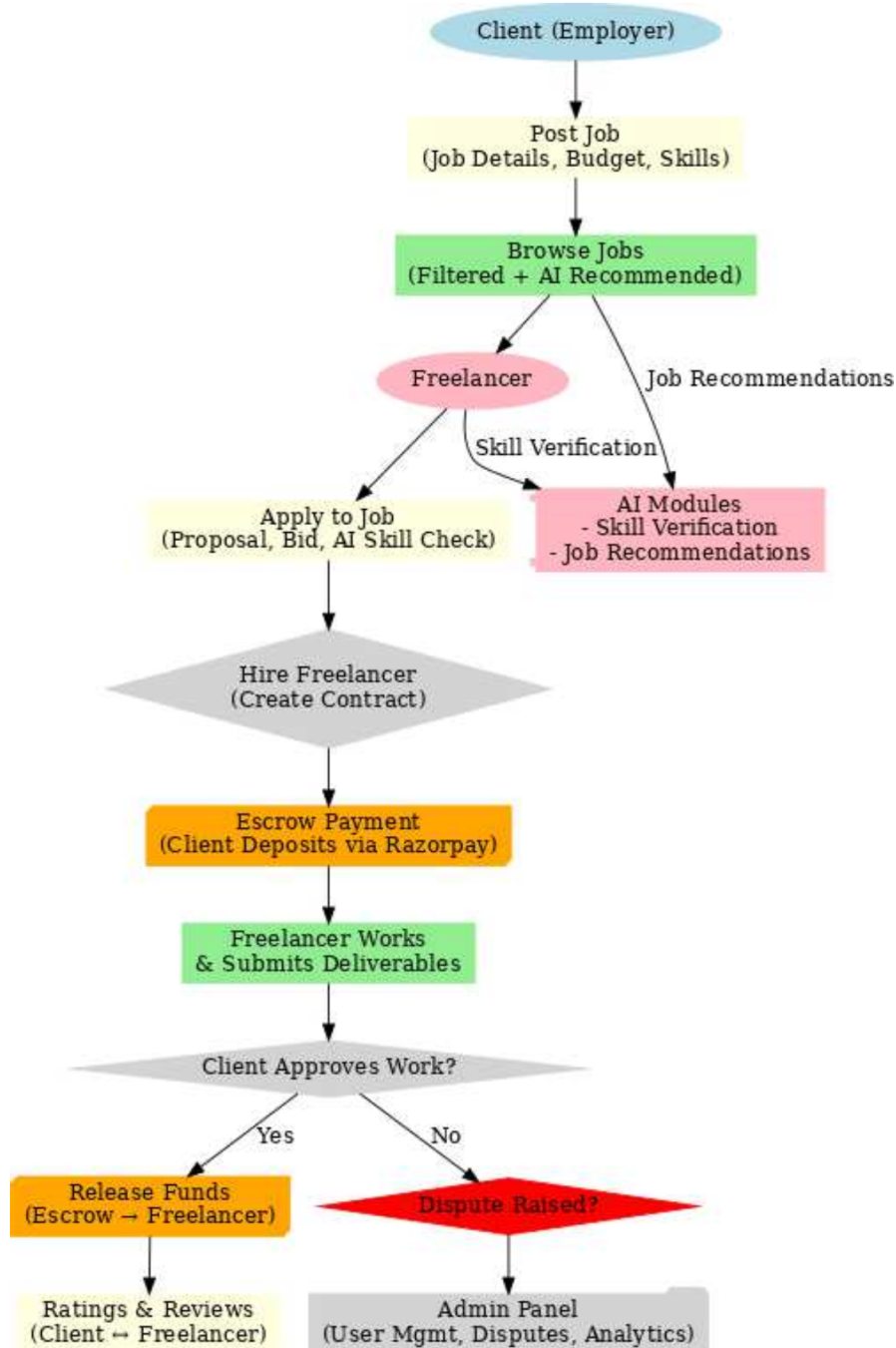


Figure 1.1: System Block Diagram for Existing System

The **Proposed System (PayShield)** introduces a trust-minimized architecture:

- **Smart Contract Escrow:** Funds are locked in a transparent contract and released only upon cryptographic verification of milestone completion.
- **Hybrid Arbitration:** To resolve conflicts, PayShield utilizes a model combining AI-driven evidence analysis with an expert human panel, ensuring faster and more consistent results than traditional legal recourse.
- **Stablecoin Settlement:** By using USDC on Layer 2 networks, the system ensures immediate liquidity and removes the volatility risks inherent in other cryptocurrencies.

1.3.1 System Architecture

The following diagram illustrates the interaction between the Freelancer, the Client, and the Smart Contract Escrow layer, including the integration of AI-arbitration and fiat on/off-ramps.

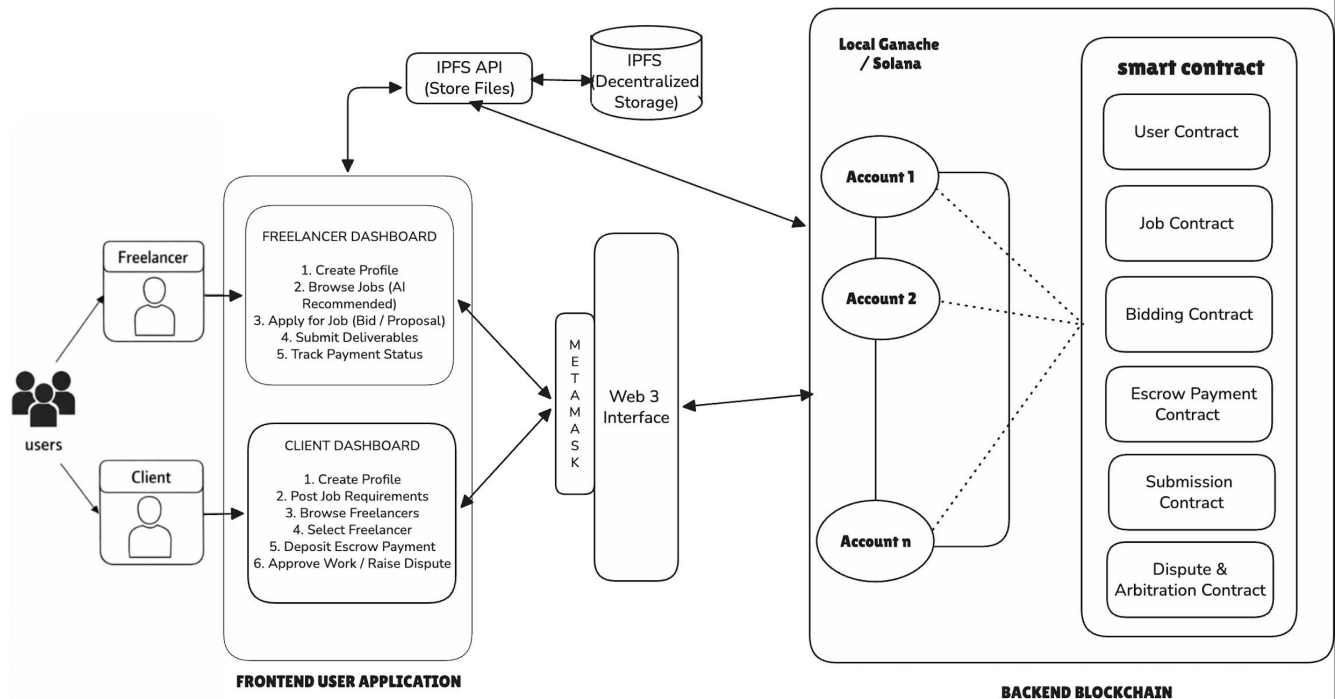


Figure 1.2: System Block Diagram for PayShield: Integrating Web3 Escrow with AI Dispute Resolution

1.3.2 System Architecture Overview

In designing **PayShield**, our team engineered a high-throughput, non-custodial decentralized application (DApp) architecture structured to facilitate secure, trustless freelance escrow payments with sub-second finality. Moving away from traditional high-fee centralized web services, we opted to build on the **Solana Blockchain** using the **Anchor Framework (Rust)**. This selection was driven by Solana's parallel transaction execution engine (*Sealevel*), extremely low transaction fees (typically less than \$0.00025), and **Program Derived Address (PDA)** capabilities, which enable deterministic and secure multi-party escrow accounts directly at the protocol level.

The PayShield architecture is organized into five tightly integrated yet decoupled layers:

1. Client Presentation and Portal Layer
2. Cryptographic Web3 and Integration Layer
3. On-Chain Protocol State Machine Layer
4. Decentralized Content-Addressable Storage Layer
5. Off-Chain AI Intelligence and Arbitration Layer

Client Presentation and Portal Layer (React / Vite)

At the frontend layer, our team developed a highly responsive **Single Page Application (SPA)** utilizing **React.js** bundled with **Vite** for rapid hot-module reloading and optimized production compilation. To overcome the static appearance commonly associated with traditional Web3 applications, we implemented a premium custom design system featuring a dynamic **SoftAurora** background and glassmorphic interface elements using modern CSS variables and reusable utility classes.

The frontend is divided into two role-based portals:

- **Client Portal:** Designed to support high-friction operations such as project creation, requirement configuration, freelancer selection, escrow initialization, deliverable reviews, dispute initiation, and fund release authorization.
- **Freelancer Portal:** Developed to support proposal submissions, milestone tracking, work pipeline monitoring, and deliverable uploads.

Both portals utilize a shared reactive state management system to provide real-time blockchain transaction feedback, including loading states, transaction confirmations, receipt generation, and animated transaction signature displays.

Cryptographic Web3 and Integration Layer

The bridge between the frontend application and the Solana network is managed using the **Solana Wallet Adapter** and custom **JSON-RPC** wrappers.

- **Decentralized Authentication:** Traditional username-password databases are completely eliminated. Authentication is performed by verifying cryptographic transaction signatures using wallet public keys from providers such as Phantom and Solflare.
- **Cryptographic Transaction Construction:** When a user initiates an action such as funding escrow, the client-side SDK gathers the required state accounts, formats transaction payloads, requests wallet signatures, serializes the transaction, and broadcasts it to the Solana network via RPC nodes.
- **SPL Token Standardization:** The payment infrastructure is built using the **Solana Program Library (SPL)** token standard with **USDC** as the settlement asset, thereby reducing cryptocurrency volatility risks.

On-Chain Protocol State Machine Layer (Solana / Anchor)

The core business logic of PayShield is implemented as a decentralized program deployed on the Solana Blockchain.

- **Program Identification:** The smart contract is deployed under the program ID: `43QYPVLRMQ9skLbbbZ3uGPsLtTbxcmuU4S5hoZ8bXJKS`
- **Program Derived Addresses (PDAs):** Instead of relying on centralized custodians, the smart contract itself securely manages escrowed funds. Unique PDA vault accounts are deterministically generated using seeds such as project identifiers, token mint addresses, and the program ID.
- **State Machine Layout:** The protocol consists of multiple state accounts including **Project**, **Escrow**, **Milestone**, **Dispute**, and **Reputation**. State transitions such as *Submitted*, *Approved*, and *Disputed* are governed by deterministic on-chain validation logic compiled into Berkeley Packet Filter (BPF) bytecode.

Decentralized Content-Addressable Storage Layer (IPFS)

Since storing large media and document files directly on-chain is economically inefficient due to Solana rent-exempt storage costs, PayShield adopts a hybrid decentralized storage architecture.

- **Content-Addressable Storage (CAS):** Large project files, deliverables, PDF briefs, code archives, and evidence documents are uploaded to the **InterPlanetary File System (IPFS)**. Each uploaded file is assigned a unique cryptographic content identifier (CID).
- **Tamper-Proof Bindings:** Any modification to a file changes its CID, making tampering immediately detectable. When a freelancer submits deliverables, the generated CID is committed on-chain through the `submit_deliverable` instruction, creating an immutable cryptographic reference between the blockchain state and the external files.

Off-Chain AI Intelligence and Arbitration Layer

To improve project matching efficiency and dispute resolution speed, PayShield integrates an off-chain AI intelligence layer interacting with both frontend and blockchain components.

- **AI Recommendation Engine:** Machine learning models extract features from job descriptions such as required skills, complexity, budget, and deadlines, and match them against freelancer reputation metrics and skill profiles stored on-chain.
- **AI Dispute Arbitrator:** During disputes, the AI engine gathers contextual information including project metadata, deliverable IPFS hashes, and communication history. It retrieves associated files from IPFS and processes them through a structured large language model (LLM) inference pipeline to generate objective dispute-resolution recommendations for human moderators.

The integration of artificial intelligence with decentralized smart contracts significantly reduces arbitration overhead, shortens resolution times, and improves transparency within freelance payment ecosystems.

2. Literature Survey and Implementation

2.1 Literature Survey Table

Title of Study	Year	Authors & Origin	Journal / Source	Research Design	Sample Size	Key Findings
AI-Powered Freelancing Platform for Escrow Payments and Skill Verification	2025	Stavros J. Benetis, Christos L. G. Ankiths, Syed Mohammed Hassan	International Symposium on Advanced Electrical and Communication Technologies	Experimental system design with modular architecture and comparative performance evaluation	Prototype evaluation conducted on sample freelancers and clients	Achieved 92.3% skill verification accuracy, 89.5% job recommendation accuracy, and 95% reduction in payment disputes through blockchain-based escrow.
Decentralized Freelancing Platforms Using Ethereum Blockchain: A Secure, Transparent Marketplace	2025	Piyush Mahajan, Divisha Aakolkar, Priyanka Dilewadkar	International Journal of Research in Engineering and Technology	System design and implementation of decentralized marketplace using Ethereum smart contracts	Prototype tested with simulated transactions	Improved transparency and trust. Reduced transaction costs and enabled secure escrow payments.
Digital Escrow and Conditional Payment Logic using Blockchain for B2B Trade	2025	Vikas Reddy Mundithi	International Journal of Web Multimedia Systems	Conceptual framework design with blockchain smart contracts	No human participants	Proposes blockchain-based escrow framework integrating smart contracts and conditional payments.

Title of Study	Year	Authors & Origin	Journal / Source	Research Design	Sample Size	Key Findings
D'Lancer: A Decentralized Freelancing Platform Using Blockchain and Smart Contracts	2025	Anjali Singh, Satyam Kumar	SCITEPRESS International Conference	System architecture and prototype implementation	Prototype system evaluation	Funds are securely stored in escrow until project completion.
Privacy-Preserving Smart Contract Solutions for Digital Supply Chains	2025	Ankush Gupta, Soumya B.	International Journal of Big Data Management	Framework design integrating blockchain and privacy techniques	Simulation-based evaluation	Introduces privacy-preserving smart contract frameworks for secure transactions.
Privacy-Preserving Smart Contracts for Permissioned Blockchains	2025	Aldino Burgos, Eduardo Alcibar	arXiv Blockchain Research	Experimental cryptographic framework design	Blockchain transaction simulation	Proposes smart contracts using zero-knowledge proofs to protect privacy.
Know Your Contract: Extending eIDAS Trust into Public Blockchains	2025	Avid Vaziry, Christoph Wrona	arXiv Research Paper	Architecture design and regulatory framework analysis	Multiple blockchain trust models	Introduces regulatory-compliant framework linking smart contracts with digital identity systems.

Table 2.1: Summary of Reviewed Literature

2.2 Implementation

The PayShield freelance platform is engineered as a production-grade, decentralized full-stack Web3 application designed to resolve trust and payment risks through a layered hybrid architecture. The platform decouples on-chain trustless execution from resource-intensive computing and data indexing. The system architecture follows a decentralized multi-tier model consisting of a presentation layer, an off-chain application layer, a decentralized storage layer, and an on-chain execution layer. This implementation is organized into six highly integrated, project-specific technical segments:

2.2.1 Frontend Application & Web3 Client Portal

The presentation layer is implemented as a high-performance React.js Single Page Application (SPA) compiled using the **Vite** bundler to minimize asset load times and optimize asset chunks. To provide a modern interface, our design utilizes custom vanilla CSS variables (`client_vite/src/index.css`) defining a dark-mode theme, a dynamic, animated **SoftAurora** background element, and glassmorphic navigation panels.

- **State Management & Performance:** Client state is optimized using React's `useMemo` and `useCallback` hooks to prevent redundant re-renders. We cache RPC calls to prevent reaching node rate-limiting boundaries.
- **Web3 Adapter Integration:** Direct ledger connectivity is achieved by embedding the `@solana/wallet-adapter-r` suite, natively supporting major non-custodial browser extensions (e.g., **Phantom**, **Solflare**). The provider is initialized in `WalletContext.jsx` targeting the `Connection` to Solana Devnet (<https://api.devnet.solana.com>).
- **Anchor Client Hook:** Custom Hook `useSolanaHubProgram.ts` imports the compiled program JSON IDL (Interface Definition Language) and constructs a read/write transaction runner:

Listing 2.1: Anchor Client Hook Implementation

```
const PROGRAM_ID = new PublicKey("43
  QYPVLRMQ9skLbbbZ3uGPstTbxcmuU4S5hoZ8bXJKS");
export const useSolanaHubProgram = () => {
  const { connection } = useConnection();
  const wallet = useAnchorWallet();
  const provider = useMemo(() => {
    if (!wallet) return null;
    return new AnchorProvider(connection, wallet, {
      preflightCommitment: "confirmed" });
  }, [connection, wallet]);
  const program = useMemo(() => {
    if (!provider) return null;
    return new Program<SolanaHubProtocol>(solanaHubJson as
      any, provider);
  }, [provider]);
  return { program, provider };
```

};

- **Responsive Layouts:** Formulated via CSS Grid and Flexbox variables, providing layout adjustments for multiple breakpoints, while interface navigation routes are managed using **React Router v7**.

2.2.2 Off-Chain Application Server Layer

The application server acts as a modular indexing middleware and metadata store, built with **Node.js** and **TypeScript** (`tsx watch src/server.ts`) powered by the **Express** framework.

- **Modular Routes & Controllers:** The server is structured into clean domain controllers:
 - `/api/auth`: Cryptographic signature login router.
 - `/api/projects`: Coordinates off-chain metadata (milestone schedules, client descriptions, file paths).
 - `/api/ai`: Houses the off-chain **AI Dispute Arbitrator** and job recommendation endpoints.
 - `/api/messaging` & `/api/social`: Manages P2P encrypted chat pathways and social networking posts.
- **Web3 Authentication:** Traditional email auth is supplemented by cryptographic signature authentication inside `/src/routes/authRoutes.ts`. Users sign a unique, server-generated message using their private key. The backend validates the signature using the **TweetNaCl** cryptographic library (`nacl.sign.detached.verify`), issuing a signed **JSON Web Token (JWT)** upon successful verification.

2.2.3 Database Architecture & Off-Chain Indexing Schema

Off-chain metadata is persisted in **MongoDB Atlas** using the **Mongoose ODM (Object Document Mapper)**. The schema bridges off-chain assets with the Solana blockchain by indexing transaction signatures and decentralized storage addresses.

- **Database Schema Mapping:**
 - **UserModel**: Maps user identities to unique Solana wallet public keys, off-chain profiles, registered skill sets, and lifetime earnings statistics.
 - **ProjectModel**: Bridges raw metadata (contract descriptions, timeline constraints) to on-chain `projectPda` addresses and records individual Solana transaction hashes (`txHash`).
 - **SubmissionModel**: Stores freelancer descriptions alongside the 46-character IPFS Content Identifiers (CIDs) and submission timestamps.
 - **DisputeModel**: Coordinates dispute tickets, recording parent milestone indices, chat message arrays, AI arbitrator split recommendations, and final settlement logs.
- **Referential Integrity:** Maintained at the application layer, ensuring records of disputes, submissions, and project parameters point back to valid registered wallet keys and on-chain program records.

2.2.4 Smart Contract Layer (Anchor/Rust Program)

The core business logic of PayShield is run securely on-chain, written in **Rust** using the **Anchor Framework**. The program is deployed to program ID 43QYPVLRMQ9skLbbbZ3uGPsLtTbxcmuU4S5hoZ8bXJKS.

- **Program Derived Addresses (PDAs):** Escrow funds, milestones, and project states are held in deterministic, seed-based PDAs. Seed schemes are enforced at the program level:
 - *Project Account:* `[b"project", project_id.to_le_bytes()]`
 - *Escrow Account:* `[b"escrow", project_id.to_le_bytes()]`
 - *Milestone Account:* `[b"milestone", project_id.to_le_bytes(), index.to_le_bytes()]`
- **Security Restrictions:** The program checks context accounts using strict Anchor validation macros:

Listing 2.2: Anchor Validation Macro Constraints

```
#[account(
    mut,
    seeds = [b"project", project_id.to_le_bytes().as_ref()],
    bump,
    constraint = project.client == client.key() @ ErrorCode::
        Unauthorized
)]
pub project: Account<'info, Project>,
```

This constraint prevents malicious accounts from calling fund release or project assignment instructions.

2.2.5 Escrow Payout, Payment Security, & Storage Integration

The financial layer implements native escrow mechanics using the **Solana Program Library (SPL) Token standard**, specifically targeting standard **USDC stablecoins** to eliminate cryptocurrency price volatility during project lifecycles.

- **Non-Custodial Escrow Vaults:** When a client initializes an escrow via `initialize_escrow`, the project budget is transferred from the client's token account to a program-derived `escrow_vault` token account. The vault is controlled solely by the `escrow` PDA authority. Payouts can only be released to the freelancer through signed milestone approvals or arbitration outcomes.
- **Auto-Release Security:** The contract protects freelancers from payment lockups. If a client goes unresponsive, the `process_auto_release` instruction releases locked funds if the `clock.unix_timestamp` $\geq$ `milestone.submitted_at + 7` days.
- **IPFS Content-Addressable Cryptographic Bindings:** Deliverables and project specifications are processed off-chain and uploaded to **IPFS** via **Pinata API** endpoints. Pinata returns an immutable 46-character cryptographic Content Identifier (CID). The client-side DApp converts this CID to a 32-byte representation (`[u8; 32]`) and writes it permanently to the on-chain `Milestone` account via `submit_deliverable`, ensuring deliverables are tamper-proof and verifiable.

2.2.6 Performance Optimization, Caching, & Administration

- **Transaction Speed:** Near-instant settlements are achieved by utilizing Solana’s sub-second block times and low transaction fees (\$0.00025), charging only a **0.5% platform fee** compared to traditional platform charges (10-20%).
- **Asynchronous IPFS Pinning:** Long-duration IPFS API network requests are handled asynchronously in the client background using `uploadToIPFS(file)`, preventing UI blocks and ensuring responsive interface performance.
- **Moderator/Admin Panel:** Express routes `/api/ai/disputes/:id/manual-override` provide platform administrators with an arbitration console. Administrators can review AI recommendation reports generated by our LLM inference module, override disputed milestones when required, and call the on-chain program’s `cast_vote` instruction to resolve disputes.

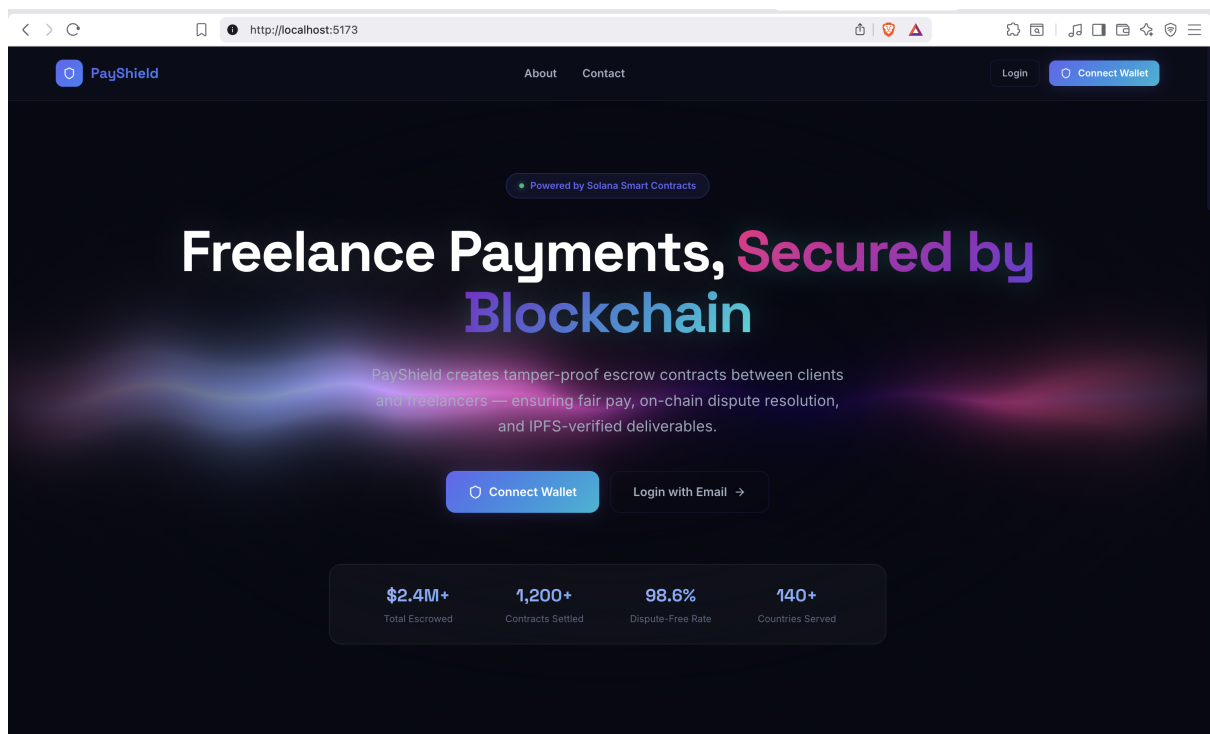


Figure 2.1: Landing page of PayShield

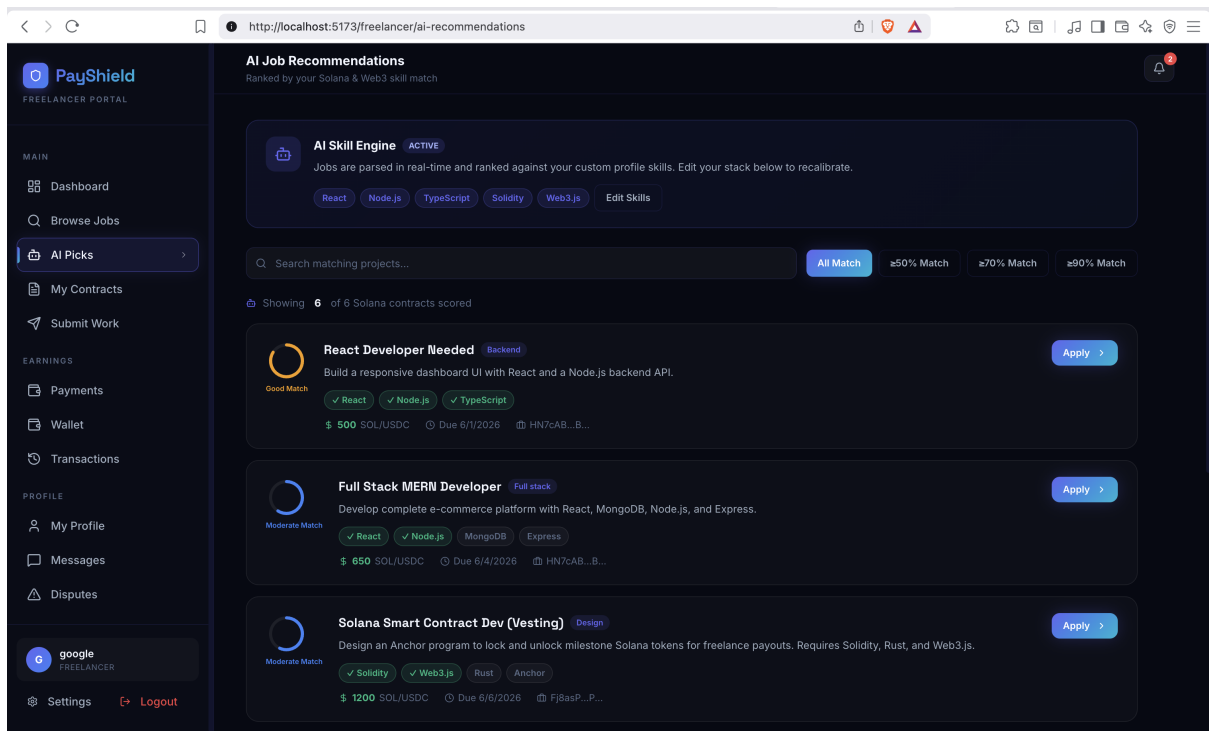


Figure 2.2: Job Prediction page of PayShield

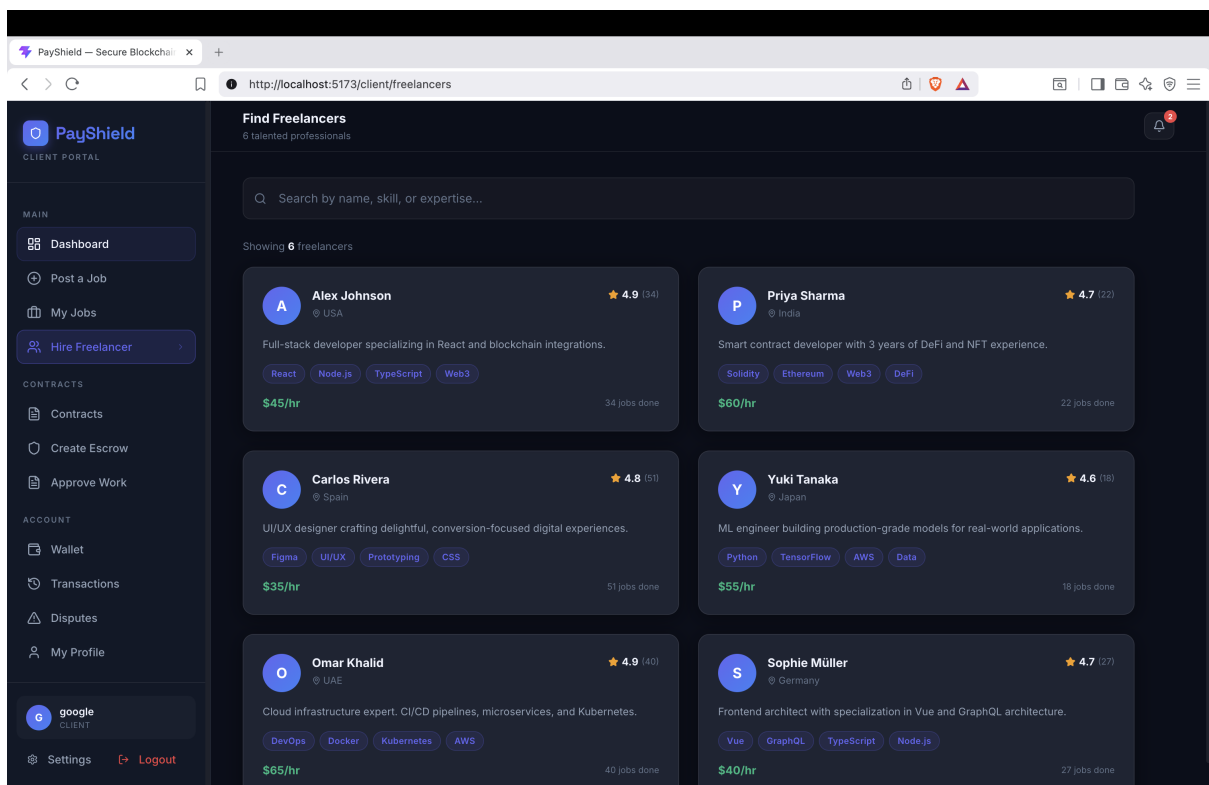


Figure 2.3: Client Dashboard page of PayShield

3. Software Requirements and Frontend Development

3.1 Tech Stack Used

3.1.1 Software Requirements

Smart Contract Development and Backend

- **Ethereum Blockchain** – Used as the decentralized platform for deploying smart contracts and executing secure transactions.
- **Solidity** – Programming language used to develop smart contracts for job posting, bidding, agreements, and payment escrow.
- **Hardhat** – Development framework used for compiling, testing, and deploying smart contracts.
- **MetaMask** – Ethereum wallet used for user authentication and secure interaction with smart contracts.

Frontend Development

- **React.js** – Used to build an interactive and responsive user interface for the marketplace.
- **Web3.js** – JavaScript library that enables communication between the frontend application and the Ethereum blockchain.
- **NPM (Node Package Manager)** – Used for managing project libraries and dependencies.
- **CSS** – Used for styling and enhancing the visual appearance of the user interface.

Security

- **JWT (JSON Web Token)** – Used for authentication and secure session management.
- **bcrypt** – Used for password hashing and encryption.
- **HTTPS Protocol** – Ensures secure data transmission between client and server.

Development Tools

- **VS Code** – Primary code editor used for development.
- **Git and GitHub** – Used for version control and collaborative development.

- **Postman** – Used for testing backend APIs.
- **Node Package Manager (npm)** – Used for installing and managing project dependencies.

3.1.2 Hardware Requirements

For Users

- Computer / Laptop / Mobile device
- Minimum 4 GB RAM (8 GB recommended)
- Intel i3 / AMD Ryzen 3 or higher processor
- Stable internet connection
- Web browser (Chrome / Edge / Firefox)

For Server

- Processor: Intel i5 / AMD Ryzen 5 or higher
- RAM: Minimum 8 GB (16 GB recommended)
- Storage: 256 GB SSD or higher
- Internet: High-speed and reliable network
- Cloud-based or on-premise server

3.2 Operating System Environment

3.2.1 Hardware Requirements

- Minimum 4 GB RAM (8 GB recommended)
- Multi-core processor (Intel i3 / AMD Ryzen 3 or higher)
- At least 256 GB storage

3.2.2 Software Requirements

- Operating System: Windows / Linux / macOS
- Node.js (latest stable version)
- React.js framework
- Solidity compiler
- Hardhat development environment
- Web3.js library

4. Conclusion and Future Scope

4.1 Conclusion

PayShield successfully demonstrates a secure and transparent freelance payment platform that combines blockchain technology, smart contracts, and modern web development to improve trust between clients and freelancers. The system addresses major issues present in traditional freelancing platforms such as payment fraud, delayed transactions, lack of transparency, and dependency on centralized intermediaries.

By integrating smart contract-based escrow mechanisms, the platform ensures that funds are released only after successful completion of project milestones. The use of blockchain technology provides tamper-proof transaction records, enhanced security, and improved reliability. Additionally, the frontend interface developed using modern web technologies offers a user-friendly and responsive experience for both clients and freelancers.

Overall, the project achieves its objective of creating a decentralized, secure, and efficient payment ecosystem for freelance services while reducing risks associated with online transactions and dispute management.

4.2 Future Scope

The future scope of PayShield includes several enhancements to improve scalability, security, and user experience. The platform can be extended to support multiple blockchain networks and cryptocurrencies for wider adoption and interoperability. Advanced AI-based fraud detection and smart recommendation systems can also be integrated to improve security and automate freelancer-client matching.

Future development may include implementation of decentralized dispute resolution, reputation-based ranking systems, and real-time analytics dashboards for better platform management. Mobile application support can also be added to provide accessibility and ease of use across different devices.

Furthermore, integration with cloud infrastructure and stronger encryption techniques can improve system performance, scalability, and data protection. These future improvements will help PayShield evolve into a more reliable, intelligent, and globally accessible freelancing payment platform.

4.3 References

1. AI-Powered Freelancing Platform with Escrow Payments and Skill Verification, Proceedings of the International Symposium, IEEE Xplore, 2025.
2. “Decentralized Freelancing Platforms Using Ethereum Blockchain: A Secure, Transparent, and Autonomous Marketplace,” International Journal of Innovative Research in Technology (IJIRT), May 2025.
3. “Resolving Smart Contract Disputes Through Blockchain Arbitration,” Kluwer Law International,

2024.

4. "Blockchain Arbitration: Roadmap to Recognition and Enforcement of Arbitral Award," Cogent Social Sciences, Taylor & Francis, 2025.
5. "DLancer: A Decentralised Freelancing Platform," Proceedings of SciTePress Conference Publications, January 2025.
6. "An Arbitrator's Toolkit: Blockchain, Cryptocurrency, and Smart Contract Dispute Resolution," Jus Mundi Legal Research, June 2025.
7. "Role of Blockchain Technology in Recruitment and Managing Gig Economy Workers," IGI Global / ResearchGate Publication, October 2025.